
Kronecker Generative Networks: A General Neural Architecture for Parameter-Efficient Learning Across Classification Tasks

Yang Yang¹ Zhengmin Kong¹ Yuan Liu¹ Tao Huang² Wei Xiang³

Abstract

Modern neural networks derive much of their effectiveness from rich connectivity patterns. Yet, existing architectures often fix the topology at either the sparse or dense extremes, thereby limiting structural flexibility and analysis. We propose Kronecker Generative Networks (KGNs), an algebraic framework that constructs neural network topologies via recursive generation rules, treating topology as a first-class design object. KGNs generate families of directed acyclic graphs with controllable connectivity complexity, enabling systematic interpolation between sparse and dense aggregation regimes. Under this formulation, architectures such as FractalNet and DenseNet arise as specific instantiations corresponding to different generation rules. We provide theoretical analysis of acyclicity, connectivity scaling, and expressiveness, and demonstrate experimentally that KGN instantiations achieve favorable accuracy-efficiency trade-offs across multiple domains.

1. Introduction

With the widespread deployment of modern neural networks, architectural design has become a central factor in achieving strong performance under practical constraints. Across a wide range of influential architectures, including residual (He et al., 2016), densely connected (Huang et al., 2017), and other multi-branch designs (Szegedy et al., 2015; Larsson et al., 2017), performance gains are often associated with increasingly rich connectivity patterns built from relatively simple components. In practice, these designs in-

volve multiple parallel pathways and non-trivial interactions across layers, extending beyond what can be expressed by simple sequential stacking. These phenomena address the underlying importance of studying the structures of neural networks as first-class algebraic objects.

The importance of network topology in neural computation has been recognized since the earliest theoretical models of neurons. In particular, McCulloch & Pitts (1943) modeled neural structures as algebraic objects, yielding two invaluable insights for deep learning: (i) neural topologies determine the behavior of neurons, up to equivalence, and (ii) the learning of neurons seems to require “permanent alternations” of net structures. These conclusions provide a biological perspective on the equivalence and expressiveness of neural networks across different topologies and suggest the inevitability of improving network architectures to enhance the learning capacity of neural networks.

Under the same mindset, residual skips (He et al., 2016) stand out as a representative approach to improve model learning with minimal structural alterations (Szegedy et al., 2017; Santhanam & Davis, 2019). Later work extends this approach by introducing more complex topologies built from existing building blocks, such as fully connected layers and convolutional blocks. Notably, FractalNet (Larsson et al., 2017) applies a complicated topology via a fixed fractal expansion rule to form an ultra-deep network that is topologically self-similar. Another exemplary work is DenseNet (Huang et al., 2017), followed by CondenseNet (Huang et al., 2018). In DenseNets, the features from each convolutional block are passed to all subsequent layers, forming a set of densely connected pathways. Compared with DenseNet and its variants, FractalNet is deeper and has more sparse shortcuts. Additionally, Huang et al. attributed DenseNet’s success to the “collective knowledge” of the final classifier, thereby underscoring the importance of network structures beyond node implementations.

While both FractalNet and DenseNet emphasize the importance of rich, organized connections, there remains room for generalization and further extension of these architectures. A key question is whether they are flexible and theoretically well-justified to describe a family of networks sharing the same characteristics. We consider the aver-

¹School of Robotics, Wuhan University, Wuhan, Hubei, China
²College of Science and Engineering, and Centre for AI and Data Science Innovation, James Cook University, Cairns, QLD 4878, Australia ³School of Computing, Engineering and Mathematical Sciences, La Trobe University, Melbourne, VIC 3086, Australia. Correspondence to: Zhengmin Kong <zmkong@whu.edu.cn>.

age in-degree of the induced directed acyclic graph, which reflects the amount of information each node aggregates. Intuitively, a higher average in-degree corresponds to more substantial reliance on collective information (Raghu et al., 2017; Xie et al., 2017) and is typically associated with increased computational overhead. Let $\langle d \rangle_F$ and $\langle d \rangle_D$ denote the average in-degrees of FractalNet and DenseNet, respectively. With minimal computation¹, it can be proved that $\langle d \rangle_F = O(1)$, $\langle d \rangle_D = O(N_D)$, where N_D denotes the node count in DenseNet. We observe that the average in-degree of FractalNet is independent of every structural parameter, while that of DenseNet grows at the same pace as node count. Consequently, FractalNet and DenseNet occupy two extreme connectivity regimes: bounded-degree aggregation, independent of network scale, and dense aggregation that scales linearly with the number of nodes. However, fixing the average in-degree at either extreme implicitly hard-codes how information is aggregated throughout the network, leaving little flexibility to modulate the complexity of connectivity between sparse and dense regimes under task-specific or computational constraints. This observation motivates the need for a framework that enables controlled exploration of intermediate topological structures.

To this end, we propose *Kronecker Generative Networks* (KGNs), a principled algebraic framework for constructing neural network topologies through recursive generation rules. KGNs treat network topology itself as a first-class design object, enabling systematic control over connectivity patterns, information aggregation, and structural complexity. By adjusting a small set of algebraic generators, KGNs produce families of directed acyclic graphs (DAGs) that interpolate smoothly between sparse and dense aggregation regimes while preserving modular compositional structure. Under this formulation, existing architectures such as FractalNet and DenseNet can be interpreted as specific instantiations of the proposed generative process. In particular, their differing connectivity behaviors correspond to distinct choices of generation rules, leading to fundamentally different scaling laws for information aggregation.

Beyond enabling flexible architecture construction, the proposed formulation enables the analysis and comparison of neural network structural properties at the level of topology. By expressing connectivity patterns through explicit algebraic generators, KGNs enable principled, architecture-agnostic reasoning about expressiveness, information aggregation, and complexity. This perspective shifts the study of network design from enumerating model-specific wiring patterns to analyzing families of networks under shared structural rules.

Overall, our contributions are listed as follows:

- (i) We introduce an algebraic recursive framework for neural topology generation that generalizes existing architectures.
- (ii) We provide a theoretical analysis of acyclicity, complexity, and expressiveness of generated networks, demonstrating the flexibility of balancing computational cost and functional capacity with topological parameters.
- (iii) We develop practical KGN instantiations with favorable accuracy-efficiency trade-offs across domains, achieving a considerable accuracy rate with fewer parameters compared with the state-of-the-art (SOTA) models in image classification and human activity recognition tasks.
- (iv) We present supporting analyses, including ablation studies, average in-degree characterization, unification results, and a generic KGN-based architecture search formulation.

Code availability The code for our implementation is open-source and publicly available on GitHub².

2. Related Work

We focus on prior works that employ Kronecker algebras and sparse topology in neural network design, as structured connectivity architectures such as FractalNet and DenseNet are discussed in the Introduction and evaluated experimentally.

Kronecker-based graph generation Kronecker products have been studied as a principled mechanism for generating large graphs, as recursive Kronecker powers lead to exponential growth in the number of nodes while preserving local connectivity patterns. In particular, Leskovec et al. (2010) define Kronecker graphs via repeated Kronecker powers of an initiator adjacency matrix and observe emergent self-similarity. While such constructions inspire fractal-like neural architectures, classical Kronecker graph models operate on abstract graphs without computational or learning semantics. Our work draws inspiration from their algebraic construction. Still, it departs fundamentally by introducing $(\tilde{P}, C)$ -inducers that generate directed acyclic computation graphs with explicit information flow, learnable components, and controllable connectivity beyond strict self-similarity.

Parameterization and decomposition Kronecker algebra has also been widely used for structured parameterization and operator decomposition in neural networks. Representative examples include Kronecker-based weight decompositions yielding adaptive activation functions (Jagtap et al.,

¹See Appendix E.

²<https://github.com/EastLakeFish/Kronecker-Generative-Networks>

2022), coefficient decompositions for improved statistical efficiency (Feng & Yang, 2022), and Kronecker-structured attention operators for computational savings (Gao et al., 2020). These works operate at the level of parameters or operators under a fixed computation graph. In contrast, our work employs Kronecker algebra to directly generate families of computation graphs, treating network topology itself as the primary design object.

Sparse Topology Recent studies have also highlighted the importance of a learnable sparse topology beyond conventional dense-to-sparse training paradigms. In particular, lottery-ticket-style methods (Frankle & Carbin, 2019) investigate trainable sparse subnetworks within overparameterized architectures, while dynamic sparse training methods such as RigL (Evci et al., 2020) optimize sparse topology during training through iterative connectivity updates. In contrast, KGN does not derive sparse structures from dense parent networks or dynamically rewire connections during training; instead, it focuses on the direct generation of neural topologies through recursive Kronecker construction rules, providing a complementary perspective centered on topology construction rather than topology selection.

3. Theoretical Grounding

We introduce the formal definition of Kronecker Generative Networks (KGNs) and analyze the topological properties induced by the generation rule. These properties provide foundations ensuring that the Kronecker-based construction yields well-formed and trainable computation graphs.

Nodes and graphs We first introduce the notation and basic graph concepts used throughout our work. In the context of KGN, a node is an atomic topological unit. Each node represents an abstract computational module, whose internal implementation may consist of multiple operations and differ across nodes. A directed graph consists of a set of nodes and directed edges between them. Formally, it is expressed by a pair $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{v_i\}_{i=1}^n$ represents the set of n vertices, or nodes. $\mathcal{E}$ corresponds to a set of ordered pairs, or edges, denoted by (v_i, v_j) , where v_i and v_j are nodes in $\mathcal{V}$. For each graph $\mathcal{G}$, an adjacency matrix $P = [p_{ij}] \in \{0, 1\}^{n \times n}$ is employed to describe the connections:

$$p_{ij} = \begin{cases} 1, & \text{if } (v_i, v_j) \in \mathcal{E}, \\ 0, & \text{otherwise,} \end{cases} \quad v_i, v_j \in \mathcal{V}. \quad (1)$$

A directed graph is acyclic if and only if its adjacency P is nilpotent. Lemma 3.1 formalized this fact.

Lemma 3.1 (Acyclicity and Nilpotency). *Let P be the adjacency matrix of a directed graph $\mathcal{G}$. The following statements are equivalent: (i) $\mathcal{G}$ is acyclic; (ii) P is nilpotent, i.e.,*

there exists $k \leq n$ such that $P^k = 0$; (iii) $P^m = 0$ for all $m \geq n$.

Lemma 3.1 is a classic result in directed graph theory. For any DAG, there exists a permutation matrix S that orders the vertices topologically. Under this ordering, the permuted adjacency matrix, i.e., $S^\top P S$, is strictly upper-triangular. This observation is equivalent to the nilpotency characterization in Lemma 3.1. In practice, neural networks are almost always acyclic, as acyclicity is required for backpropagation to be well-defined.

For clarity of presentation, we assume that the nodes in every DAG we study are already topologically sorted, as formalized by Definition 3.2.

Definition 3.2 (Topological Sorting). Given any DAG $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, it is topologically sorted if its adjacency matrix P is strictly upper-triangular.

Next, we establish KGN based on a Kronecker-based generation rule, as shown in Definition 3.3.

Definition 3.3 (KGN Transform). Let

$$\{n_i \in \mathbb{Z}_+\}_{i=0}^k \quad (2)$$

denote node counts. Let $\tilde{P}_i$ be strictly upper-triangular matrices, and C_i be arbitrary binary adjacency matrices, where

$$\begin{aligned} 0 \neq \tilde{P}_i &\in \{0, 1\}^{n_i \times n_i}, \quad i = 0, 1, \dots, k, \\ 0 \neq C_i &\in \{0, 1\}^{\tilde{n}_{i-1} \times \tilde{n}_{i-1}}, \quad i = 1, 2, \dots, k, \\ \tilde{n}_{i-1} &= n_{i-1} \cdot \tilde{n}_{i-2}, \quad \tilde{n}_{-1} := 1. \end{aligned}$$

The KGN transform is defined as

$$\begin{aligned} P_i &= \mathcal{T}_i(P_{i-1}) := I_{n_i} \otimes P_{i-1} + \tilde{P}_i \otimes C_i, \\ P_0 &= \tilde{P}_0, \quad i = 1, 2, \dots, k, \end{aligned} \quad (3)$$

where $\otimes$ denotes the Kronecker product. I_{n_i} is the $n_i \times n_i$ identity matrix.

Intuition The KGN transform consists of two components: a duplication term $I_{n_i} \otimes P$, which replicates the previous graph structure n_i times, and a connection term $\tilde{P}_i \otimes C_i$, which links the replicated structures according to the inducer $\tilde{P}_i$ and connector C_i .

Definition 3.4 (Kronecker Generative Network). Let $d \in [1, k]$, P_d be an adjacency matrix generated by Definition 3.3, the directed graph

$$\mathcal{G}_d = (\mathcal{V}_d, P_d) \quad (4)$$

is defined as a depth- d KGN. Adjacency matrices $\tilde{P}_d = \{\tilde{P}_i\}_{i=0}^d$ and connector matrices $C_d = \{C_i\}_{i=1}^d$ are the inducers of $\mathcal{G}_d$.

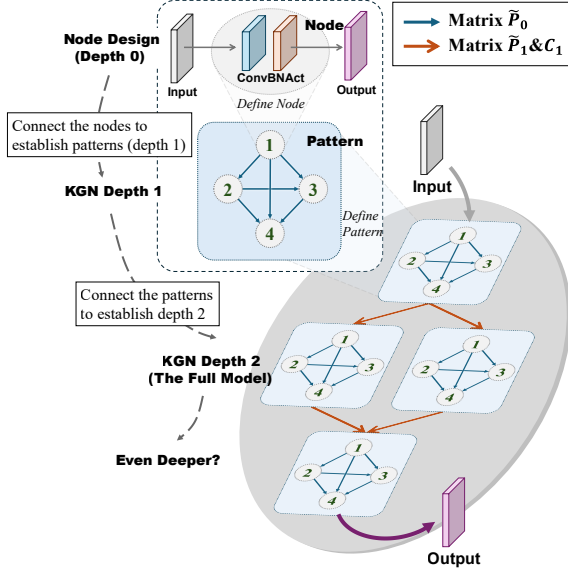


Figure 1. A depth-2 16-node KGN.

That is, a KGN is a neural network whose topology is defined by (3). A depth-2 KGN is shown in Figure 1 as a toy example. It is induced by $\tilde{P}_0$, $\tilde{P}_1$ and connector C_1 . Using Definition 3.3, we can recursively define deeper KGNs.

Unification Property 3.5 highlights a fundamental distinction between existing architectures and KGN: while DenseNet and FractalNet rely on fixed connectivity rules, KGN exposes topology generation itself as a controllable design dimension.

Property 3.5 (Unification of FractalNet and DenseNet). Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a KGN. The following statements hold:

- (i) **DenseNet.** $\mathcal{G}$ generalizes to a DenseNet if $\tilde{P}$ is chosen as a fully connected upper-triangular adjacency matrix, and $C = \mathbf{1}\mathbf{1}^\top$.
- (ii) **FractalNet.** $\mathcal{G}$ generalizes to a FractalNet using the following inducers:

$$\begin{aligned} \tilde{P}_i &= \begin{bmatrix} 0 & \mathbf{1} \\ 0 & 0 \end{bmatrix}, \quad i = 0, 1, \dots, C, \\ C_i &= E_i + E_i^\top, \quad i = 1, 2, \dots, C. \end{aligned} \quad (5)$$

Here E_i denotes a matrix of proper shape with the only nonzero entry at $(1, |\mathcal{V}_{i-1}|)$.

Property 3.5 positions DenseNet and FractalNet as structurally constrained realizations within the broader KGN framework. A remark on Statement (ii) is provided in Appendix A. KGN treats topology as a generative object rather than a fixed construction, allowing connectivity patterns to be systematically varied while preserving acyclicity and structural interpretability.

Acyclicity Starting with a property of the Kronecker product, we show that KGNs at depth $d > 0$ are always acyclic, so that backpropagation can be well-defined.

Lemma 3.6 (Block Triangularity). *Let $P \in \mathbb{R}^{n \times n}$ be a strictly upper-triangular matrix. The Kronecker lift $P \otimes C$ is also strictly upper-triangular for an arbitrary matrix C of compatible shape.*

Lemma 3.6 implies that Kronecker-based generation preserves the triangularity of matrices. The acyclicity of KGNs is then implied by Property 3.7.

Property 3.7 (Acyclicity of KGNs). Given $d > 0$ and KGN $\mathcal{G}_d$ generated by (3), the following statements hold:

- (i) (Sufficiency) $\mathcal{G}_d$ with adjacency matrix P_d is topologically sorted and acyclic, if every $\tilde{P} \in \tilde{\mathcal{P}}$ is strictly upper-triangular.
- (ii) (Necessity) If $\mathcal{G}_d$ is acyclic, every $\tilde{P} \in \tilde{\mathcal{P}}_d$ is nilpotent.

The proof of Lemma 3.6 and Property 3.7 is given in Appendix A, which is directly established by Lemma 3.1, 3.6, and the properties of the Kronecker product. Based on Property 3.7, we can also conclude intuitively that when KGNs are built upon structures where backpropagation is well-defined, they can always be trained using algorithms based on backpropagation.

Property 3.8 (Complexity). Let $\mathcal{G}_d = (\mathcal{V}_d, \mathcal{E}_d)$ denote a depth- d KGN. For any $\{0, 1\}$ -adjacency matrix A , define

$$\|A\|_{1,1} = \sum_{i,j} A_{ij} = \mathbf{1}^\top A \mathbf{1}. \quad (6)$$

Also define $\|C_0\|_{1,1} = 1$. The following statements hold:

- (i) The number of nodes is $|\mathcal{V}_d| = \prod_{i=0}^d n_i$.
- (ii) The number of edges satisfies

$$|\mathcal{E}_d| = \sum_{i=0}^d \left(\prod_{t=i+1}^d n_t \right) N_i,$$

where $N_i = \|\tilde{P}_i\|_{1,1} \cdot \|C_i\|_{1,1}$.

The proof of Property 3.8 follows directly from the recursive construction in (3) and is given in Appendix A. A useful consequence is that the size of a KGN is governed entirely by the inducers and the generation depth d : the node count satisfies $|\mathcal{V}_d| = O(\max_{0 \leq i \leq d} n_i)$ and $|\mathcal{V}_d| = \Omega(\min_{0 \leq i \leq d} n_i)$. When nodes share comparable parameters and compute cost, the overall model size and inference time follow directly from the structural complexity:

$$\text{Param}_{\text{KGN}} \approx |\mathcal{V}_d| \cdot \text{Param}_{\text{node}}, \quad (7)$$

$$T_{\text{KGN}} \approx |\mathcal{E}_d| \cdot T_{\text{node}}, \quad (8)$$

where $\text{Param}_{\text{node}}$ and T_{node} denote the typical parameter count and compute duration of a single node, respectively.

Thus, Property 3.8 provides a direct handle for estimating and controlling the resource footprint of KGN architectures by choosing inducers and depth.

Property 3.9 (Parameter Reuse via Path Aggregation). Let $\mathcal{G}_d$ be a KGN of depth d . Each node in $\mathcal{G}_d$ receives information from an average of

$$\langle d \rangle = \sum_{i=0}^d \frac{N_i}{\prod_{t=0}^i n_t} \quad (9)$$

incoming edges. Consequently, the average count of distinct computational paths terminating at a node grows algebraically with the sequence $\{N_i\}_{i=0}^d$.

Property 3.9 is a straightforward consequence of Property 3.8, which shows that the in-degree in KGNs reflects a structured growth of path aggregation and, therefore, feature reuse across generations. As N_i grows, deeper nodes receive information from more upstream representations, enabling richer feature mixing without proportionally increasing parameters.

Expressiveness The expressiveness of a neural topology is defined as the family of functions it can represent under all possible choices of node operations. Let $\mathcal{F}_A$ and $\mathcal{F}_B$ denote the function families induced by two networks $\mathcal{F}_A$ and $\mathcal{F}_B$, respectively; $\mathcal{F}_A$ is more expressive if $\mathcal{F}_B \subseteq \mathcal{F}_A$.

Among DAGs with the same number of nodes, a fully connected topology is maximally expressive, as it can simulate any sparser DAG by disabling unused connections. Accordingly, KGNs achieve maximal expressiveness when induced by dense adjacency matrices.

Property 3.10 (Bound of In-Degree). The average in-degree of KGNs has a growth rate of $O(\log n)$ if each inducer matrix has (i) at least 2 nodes, i.e., $\tilde{P}_i, C_i \in \mathbb{R}^{\geq 2 \times 2}$, and (ii) bounded node count, or equivalently $\|\tilde{P}_i\|_{1,1}, \|C_i\|_{1,1} < \infty$.

Property 3.10 is derived from Properties 3.8 and 3.9, showing that the average in-degree of KGNs grows between the $O(1)$ and $O(n)$ regimes in many practical cases, highlighting the flexibility of balancing information aggregation and parameter overhead. A rigorous proof of this property is given in Appendix A.

Property 3.11 (Fully Connected KGN). A KGN induced by $\tilde{P}$ and C is fully connected if (i) for all $\tilde{P} = [\tilde{p}_{ij}] \in \tilde{\mathcal{P}}$, $\tilde{p}_{ij} = 1$ if $i < j$, and (ii) for all $C \in \mathcal{C}$, $C = \mathbf{1}\mathbf{1}^\top$.

Property 3.11 follows directly from (3) and recovers DenseNet (Huang et al., 2017) as a special case. However, as shown in Property 3.8, dense KGNs incur substantial computational cost. In practice, sparse or structured choices of $\tilde{P}_i$ and C_i are therefore preferred to balance expressiveness, efficiency, and trainability.

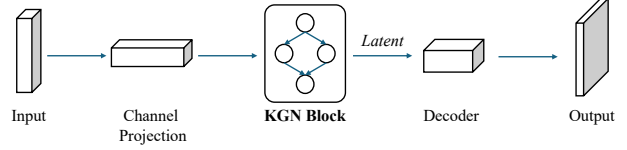


Figure 2. KGN-based encoder-decoder architecture.

Architecture search The Kronecker-based rule yields a structured architecture space in which node functions, inducers, and connector patterns are all explicit design choices. This enables both manual design and automated architecture search under clear acyclicity and complexity constraints. In this work, we focus on manually designed KGNs to isolate architectural effects; a general NAS formulation and comparisons of different model topologies are presented in Appendix C.

4. Methodology

According to (3), KGN nodes can have multiple input branches. In this section, we present two necessary structures that fuse arbitrary input branches of different shapes into a unified tensor. We also introduce a practical, hierarchical construction plan for building KGN structures, which will be implemented by designing four lightweight models for image and sequence classification in Section 5.

4.1. Fusion Layers

To deal with input branches of different shapes (e.g., $16 \times 3 \times 3$ v.s. $8 \times 16 \times 16$), a simple yet effective workflow is employed to fuse the information in the inputs: 1) The input features are preprocessed to a shared shape depending on the fusion type; 2) The preprocessed features are fused by concatenation or summation; 3) The fused features are used to feed a projection to yield an output of the desired shape.

Specifically, we extended two widely used fusion structures and adapted them to multi-input KGN nodes, yielding a KGN fusion framework. These modules are known as Inception-Styled Fusion (ISF), Residual-Style Fusion (RSF), and Biased RSF. Due to the limit of the length of the paper, we leave the detailed introduction to Appendix B.

4.2. KGN-Based Encoder-Decoder

To fully adapt KGNs to different tasks, we propose the encoder-decoder architecture based on KGNs. This architecture is shown in Figure 2.

Encoder KGN blocks are shallow instantiations of the KGN generation rule (3), typically consisting of at least one depth-2 KGN. The encoder is built by stacking pre-processing projections and KGN blocks. In the encoder,

preprocessing projections first map the input features to the permissible input space of KGN blocks. KGN blocks then extract the multiscale features required by the decoder.

Decoder Given the final KGN representation, the decoder maps the top-level features into task-specific outputs. For classification, this reduces to a global aggregation followed by a small projection head; for regression or sequence labeling tasks, the decoder produces outputs in the required temporal or spatial format. Importantly, the decoder does not impose structural constraints on the encoder: Any KGN topology that maintains well-defined feature shapes can be paired with a compatible decoder.

Design principle The encoder–decoder abstraction allows KGNs to serve as drop-in replacements for conventional backbones while retaining the mathematical guarantees of acyclicity, controllable complexity, and topological expressiveness. KGNs provide the combinatorial and multiscale structure, whereas the decoder ensures that the resulting features are aligned with downstream tasks. This modularity also enables systematic exploration of architectures (Algorithm 1), since the encoder topology can vary independently of the decoding scheme.

KGN-based encoder-decoder architecture bridges the gap between KGNs as theoretical structures and practical models for various tasks. More details are provided in Section 5.

5. Experiments

We evaluate KGNs on two settings, image classification and sequence classification, using CIFAR-10/100 (Krizhevsky et al., 2009), ImageNet-1K (Deng et al., 2009), UCI-HAR (Anguita et al., 2013), and PAMAP2 (Reiss & Stricker, 2012). Ablation studies further isolate the contribution of the KGN topology.

To assess both predictive performance and architectural compactness, we employ an accuracy score:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\hat{y}_i = y_i). \quad (10)$$

Accuracy measures the global proportion of correctly predicted samples and reflects overall classification fidelity.

5.1. Image Classification

We evaluate KGN on image classification benchmarks using KGN-Helix5, a two-layer KGN with 20 lightweight atomic nodes. Three variants are considered: Helix5-C10-S, Helix5-C10-L, and Helix5-C100, where the CIFAR-10 and CIFAR-100 models differ only in output dimensionality, and Helix5-C10-S adopts a lightweight node configuration.

Topology We developed KGN-Helix5, a two-layer KGN architecture with the following inducers:

$$\tilde{P}_{0,ij}^{\text{h5}} = \begin{cases} 1, & (i,j) \in \{(1,2), (1,3), (2,4), (3,4)\}, \\ 0, & \text{otherwise}, \end{cases} \quad (11)$$

$$\tilde{P}_{1,ij}^{\text{h5}} = \begin{cases} 1, & 1 \leq i < j \leq 5 \text{ and } (i,j) \neq (1,4), \\ 0, & \text{otherwise}. \end{cases} \quad (12)$$

The connection matrices are defined as square matrices of proper shape whose only non-zero entry is at $(|\mathcal{V}_d|, 1)$. The design of the Helix5 architecture reflects our ambition to balance expressiveness and computational cost, following the principle discussed in Property 3.11. We choose a sparse inducer $\tilde{P}_0^{\text{h5}}$ for the first layer, and a dense yet not fully connected inducer $\tilde{P}_1^{\text{h5}}$ for the second layer. Thus, each topological node functions as a shallow yet effective feature extractor, while together they form an expressive model powered by KGN’s topology design, as in (3). We employ the KGN-based encoder-decoder architecture for our models, as discussed in Section 4.2. The visualization of Helix5 structure is shown in Figure 9, Appendix D.

Node planning For the ease of implementation, we designed the Self-Contained Expansion-Fusion (SCEF) structure for every atomic node following the three-level topology specified by $\tilde{P}_0^{\text{h5}}$. The structure is shown in Appendix D, Figure 10. The four nodes are implemented as convolution blocks, and their detailed structures vary across different Helix5 architectures, as shown in Appendix D, Table 7.

Training settings For CIFAR-10 and CIFAR-100, we use the standard PyTorch dataset implementation, which provides the official 50k/10k train/test split. The validation set is formed by reserving 5k images from the original training set, following common practice. KGN-Helix5 is trained from scratch following the standard approach used in (Huang et al., 2017), using the stochastic gradient descent (SGD) optimizer (weight decay is 10^{-4} , momentum is 0.9). The data are augmented using community-standard transformations, including random flips, crops, and normalization. We use cross-entropy as the training loss, with a label smoothing rate of 0.08. The models are trained for 350 epochs with a batch size of 128. The best validation accuracy achieved during training is recorded, and the random seed is fixed to 42 for reproducibility.

Results The results on CIFAR-10 and CIFAR-100 are summarized in Table 1. FractalNet achieves competitive accuracy but relies on 22 – 39M parameters, resulting in low efficiency. More recent baselines typically operate in the 4 – 12M parameter range and achieve moderate efficiency gains, yet remain substantially larger than lightweight architectures. In contrast, KGN models, with only 0.21 – 0.41M

Table 1. Comparison of model sizes and accuracy on CIFAR-10 and CIFAR-100.*

Group	Variant	#Params	C10+ Acc. [†]	C100+ Acc. [‡]
FractalNet (Larsson et al., 2017)	Plain-20	38.6M	94.78%	76.70%
	DropPath-20	38.6M	95.40%	76.27%
	Plain-40	22.9M	94.76%	77.51%
FSTA-SNN ^{a, b} (Yu et al., 2025)	ResNet-20	11.3M	94.72%	73.44%
CPT-SNN (Xia et al., 2025)	ResNet-18	11.179M	95.79%	78.34%
	ResNet-18+DC	11.179M	95.86%	78.74%
ViT-Tiny ^c (Zhang et al., 2025)	ViT-Tiny	5.5M	96.41%	78.05%
	ViT-Tiny w/o PE	5.5M	96.32%	77.31%
Spiking Transformer (Guo et al., 2025)	ST-4-256	4.15M	94.96%	77.49%
	ST-2-384	5.76M	95.70%	78.59%
LWC ^d (Li et al., 2023)	LWC10_VB	0.39M	92.29%	–
	LWC100_VD	1.01M	–	70.45%
DenseNet (Huang et al., 2017)	DenseNet-12	1.0M	94.76%	75.58%
	DenseNet-12(BC)	0.8M	95.49%	77.73%
CondenseNet ^e (Huang et al., 2018)	CondenseNet ^{light} -94	0.33M	95.00%	75.92%
	CondenseNet-86	0.52M	95.00%	76.36%
Low-Res MobileNetV2 (Yuan et al., 2022)	All improvements	0.36M	93.32%	71.60%
KGN (Ours)	Helix5-C10-S	0.211M	95.06%	–
	Helix5-C10-L	0.373M	96.31%	–
	Helix5-C100	0.408M	–	79.55%

* The symbol “–” represents a metric value that is not applicable to the corresponding model.

^{†, ‡} Unless explicitly specified, the data are augmented by random crop and flip, following (Huang et al., 2017).

^{a, d} To the best of our knowledge, augmentation details are not provided in these papers.

^b The ResNet-19 variant proposed by the authors is not included as the parameter count is missing in the paper.

^c ViT-Tiny employed extra augmentation methods, including MixUp (Zhang et al., 2018) and CutMix (Yun et al., 2019).

^e Using the pruned version for the fairness of parameter count.

Table 2. Ablation results on CIFAR-100 using Helix5-C100.

Method	Params	Accuracy
Full Helix5	0.408M	79.55%
Chain Topology	0.314M	77.88%
Remove KGN	0.135M	64.48%

parameters, achieve accuracies comparable to or exceeding those of these much larger baselines. Consequently, KGN attains the highest parameter efficiency across all models. These results demonstrate that KGN achieves a favorable balance between accuracy and compactness through its topological design.

Ablation study We conduct ablation experiments on Helix5-C100 using CIFAR-100 to isolate the contribution of the KGN topology. First, removing all KGN-induced

nodes while retaining only the stem and final fusion node reduces the parameter count from 0.408M to 0.135M and leads to a substantial drop in accuracy from 79.55% to 64.48%. Second, constraining the Kronecker-generated connectivity to a simple feed-forward chain by retaining only first-order connections while preserving all node implementations results in a smaller but consistent performance degradation (79.55% $\rightarrow$ 77.88%). These results indicate that performance gains cannot be explained solely by node-level expressiveness or depth, but arise from the additional compositional paths enabled by the full KGN topology. The detailed results are reported in Table 2.

ImageNet To assess scalability on large-scale classification, we evaluate Helix5-ImageNet on ImageNet-1K. The model achieves 74.89% top-1 accuracy with 4.6M parameters, placing it favorably among compact architectures. Full training details and additional analyses are deferred to

Appendix F.

5.2. Sequence Classification

In addition to image classification on the CIFAR series, we further evaluate KGN on sequence classification tasks by instantiating two models, KGN-HAR and KGN-PAMAP2.

The experiments are conducted on the UCI-HAR and PAMAP2 datasets, two widely used human activity recognition benchmarks with markedly different noise characteristics and subject variability. As in the KGN-CIFAR models, both architectures follow the encoder-decoder design described in Section 4.2.

Topology KGN-HAR and KGN-PAMAP2 are two-layer KGN-based models that share the same fully connected inducers, defined in (13).

$$\tilde{P}^{\text{seq}} = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, C^{\text{seq}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}. \quad (13)$$

The visualization of structures of KGN-HAR and KGN-PAMAP2 is shown in Figure 9, Appendix D.

Node planning The nodes in KGN-HAR and KGN-PAMAP2 differ substantially, as the PAMAP2 dataset is known to be more chaotic and thus harder to fit. Unlike the CIFAR series, we designed two types of atomic nodes for sequence classification tasks, and explicitly organized the models into a two-layer, three-level (node-pattern-encoder) structure. Atomic nodes (see Section D.2) contain only convolutional layers and necessary regularization, such as batch normalization and dropout. The fusion is implemented by direct concatenation, which follows from the ISF but is more parameter-efficient for KGN structures, preserving feature shapes. Details of the implementation of both models are shown in Table 8 and 9, Appendix D.

Training settings All models were implemented in PyTorch and trained using the AdamW optimizer (Loshchilov & Hutter, 2019). For the UCI-HAR dataset, we follow the official train/test split and use multivariate inertial signals comprising tri-axial total acceleration, body acceleration, and gyroscope measurements, yielding nine input channels with a fixed sequence length of 128. For PAMAP2, we adopt a subject-independent evaluation protocol: raw IMU signals from hand, chest, and ankle sensors are preprocessed by removing invalid activities, interpolating missing values, and normalizing using only the training data statistics. The continuous signals are then segmented into non-overlapping windows of length 256, with labels assigned by majority voting within each window. Mixed-precision training with gradient scaling is enabled for all experiments to improve

numerical stability and efficiency, and a cosine learning-rate scheduler is applied throughout training. The random seed is also fixed to 42 for reproducibility.

Results Table 3 compares KGN with representative lightweight and multiscale baselines on UCI-HAR and PAMAP2. On UCI-HAR, KGN-HAR achieves 97.10% accuracy with only 0.111M parameters, outperforming most CNN-based baselines while using substantially fewer parameters. Although HS-ResNet achieves the highest absolute accuracy (97.38%), it requires more than six times as many parameters, resulting in lower parameter efficiency.

On the more challenging PAMAP2 dataset, KGN-PAMAP2 reaches 95.6% accuracy with only 0.206M parameters under a subject-independent evaluation protocol. This performance is competitive with, and in some cases exceeds, prior multiscale CNN architectures such as HS-ResNet, despite using nearly an order of magnitude fewer parameters. It is worth noting that several lightweight baselines report results under random sample-level splits, which are known to produce overly optimistic estimates on PAMAP2.

Ablation study Table 4 summarizes the impact of topology and node composition on PAMAP2. The full KGN achieves 95.6% accuracy. Rearranging its atomic nodes into a chain topology lowers accuracy to 89.9% (−5.7%), while removing the KGN module entirely yields 84.8% (−10.8%). Since the chain variant remains stronger than the model without KGN, the results indicate that both node design and the multi-branch topology contribute to performance.

6. Conclusion

We propose Kronecker Generative Networks (KGNs), a principled framework that recursively constructs network topologies via Kronecker-based generation rules. We prove that KGN-induced topologies are acyclic when initialized from acyclic inducers and analytically characterize their complexity in terms of algebraic expressions. KGNs generalize prior architectures, such as FractalNet and DenseNet, under specific inducement schemes, demonstrating the flexibility of our formulation. Extensive experiments on image and sequence classification benchmarks confirm the accuracy and parameter efficiency of our designs.

Impact Statement

This work proposes a general algebraic framework for the design of neural network topologies and poses no immediate societal risks. The framework itself is application-agnostic and can be applied across a wide range of learning tasks, including vision and sequence modeling. As with most advances in model architecture, potential downstream impacts depend on how the resulting models are deployed. We en-

Table 3. Comparison of model sizes and accuracy on UCI-HAR and PAMAP2.*

Group	Model	UCI-HAR		PAMAP2 [†]	
		#Params	Accuracy	#Params	Accuracy
CNN	DKInception (Yu & Al-Qaness, 2024)	0.398M	95.70%	0.471M	89.72% ^a
	Linear-grouped CNN (Liu et al., 2022)	0.59M	96.06%	8.83M	91.46% ^b
	Linear-grouped ResNet (Liu et al., 2022)	0.48M	95.04%	2.94M	91.50% ^b
all-MLP (Wang et al., 2024)	PatchHAR	–	–	0.6M ^c	90.82%
Hierarchical-Split (Tang et al., 2022)	HS-CNN	0.46M	96.77%	0.98M	92.58%
	HS-ResNet	0.67M	97.38%	1.20M	94.29%
KGN (Ours)	KGN-HAR	0.111M	97.10%	–	–
	KGN-PAMAP2	–	–	0.206M	95.60%

* “–” indicates that the corresponding metric is not applicable.

[†] PAMAP2 results are obtained under a subject-independent evaluation protocol (LOSO or equivalent), where no samples from test subjects appear in the training set.

^a The authors did not report whether they used a subject-independent split, or the “leave-one-out” scheme explicitly.

^b This method employs random sample-level splits on PAMAP2 rather than subject-independent evaluation.

^c We use the parameter count reported in Table II (Wang et al., 2024), as the architecture is unchanged in Table II and V. Still, the original paper does not explicitly restate model size for the “leave-one-out” setting.

Table 4. Ablation results on PAMAP2 using KGN-PAMAP2.

Method	Params	Accuracy
Full KGN	0.206M	95.6%
Chain Topology	0.171M	89.9%
Remove KGN	0.006M	84.8%

courage future work to examine these aspects in application-specific settings and to assess broader societal implications where relevant.

Acknowledgments

The authors thank the anonymous reviewers and area chairs for their valuable comments and constructive suggestions, which helped improve the presentation of this work. The work of Zhengmin Kong is supported by Smart Grid-National Science and Technology Major Project under Grant 2024ZD0802100.

References

Anguita, D., Ghio, A., Oneto, L., Parra, X., Reyes-Ortiz, J. L., et al. A public domain dataset for human activity recognition using smartphones. In *Esann*, volume 3, pp. 3–4, 2013.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei,

L. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pp. 248–255. IEEE, 2009.

Evci, U., Gale, T., Menick, J., Castro, P. S., and Elsen, E. Rigging the lottery: Making all tickets winners. In *International conference on machine learning*, pp. 2943–2952. PMLR, 2020.

Feng, L. and Yang, G. Deep kronecker network. *arXiv preprint arXiv:2210.13327*, 2022.

Frankle, J. and Carbin, M. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=rJl-b3RcF7>.

Gao, H., Wang, Z., and Ji, S. Kronecker attention networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 229–237, 2020.

Guo, Y., Liu, X., Chen, Y., Peng, W., Zhang, Y., and Ma, Z. Spiking transformer: Introducing accurate addition-only spiking self-attention for transformer. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 24398–24408, 2025.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings*

- of the *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016. doi: 10.1109/CVPR.2016.90.
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., and Adam, H. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- Huang, G., Liu, Z., Van Der Maaten, L., and Weinberger, K. Q. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4700–4708, 2017.
- Huang, G., Liu, S., Van der Maaten, L., and Weinberger, K. Q. Condensenet: An efficient densenet using learned group convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2752–2761, 2018.
- Jagtap, A. D., Shin, Y., Kawaguchi, K., and Karniadakis, G. E. Deep kronecker neural networks: A general framework for neural networks with adaptive activation functions. *Neurocomputing*, 468:165–180, 2022.
- Krizhevsky, A., Hinton, G., et al. Learning multiple layers of features from tiny images. Toronto, ON, Canada, 2009.
- Larsson, G., Maire, M., and Shakhnarovich, G. Fractalnet: Ultra-deep neural networks without residuals. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=S1VaB4cex>.
- Leskovec, J., Chakrabarti, D., Kleinberg, J., Faloutsos, C., and Ghahramani, Z. Kronecker graphs: an approach to modeling networks. *Journal of Machine Learning Research*, 11(2), 2010.
- Li, H., Yue, X., Zhao, C., and Meng, L. Lightweight deep neural network from scratch. *Applied Intelligence*, 53 (15):18868–18886, 2023.
- Liu, T., Wang, S., Liu, Y., Quan, W., and Zhang, L. A lightweight neural network framework using linear grouped convolution for human activity recognition on mobile devices. *The Journal of Supercomputing*, 78(5): 6696–6716, 2022.
- Loshchilov, I. and Hutter, F. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=Skq89Scxx>.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- McCulloch, W. S. and Pitts, W. A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 5(4):115–133, 1943.
- Raghu, M., Poole, B., Kleinberg, J., Ganguli, S., and Sohl-Dickstein, J. On the expressive power of deep neural networks. In *international conference on machine learning*, pp. 2847–2854. PMLR, 2017.
- Reiss, A. and Stricker, D. Introducing a new benchmarked dataset for activity monitoring. In *2012 16th international symposium on wearable computers*, pp. 108–109. IEEE, 2012.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4510–4520, 2018.
- Santhanam, V. and Davis, L. S. A generic improvement to deep residual networks based on gradient flow. *IEEE Transactions on Neural Networks and Learning Systems*, 31(7):2490–2499, 2019.
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., and Rabinovich, A. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1–9, 2015.
- Szegedy, C., Ioffe, S., Vanhoucke, V., and Alemi, A. Inception-v4, inception-resnet and the impact of residual connections on learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 31, 2017.
- Tang, Y., Zhang, L., Min, F., and He, J. Multiscale deep feature learning for human activity recognition using wearable sensors. *IEEE Transactions on Industrial Electronics*, 70(2):2106–2116, 2022.
- Wang, S., Zhang, L., Wang, X., Huang, W., Wu, H., and Song, A. Patchhar: A mlp-like architecture for efficient activity recognition using wearables. *IEEE Transactions on Biometrics, Behavior, and Identity Science*, 6(2):169–181, 2024.
- Xia, Q., Yu, Y., Chang, Z., Hui, B., and Luo, H. Cpt-snn: A spiking neural network that can combine the previous timestep. *Neurocomputing*, pp. 130253, 2025.
- Xie, S., Girshick, R., Dollár, P., Tu, Z., and He, K. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1492–1500, 2017.
- Yu, K., Zhang, T., Wang, H., and Xu, Q. Fsta-snn: Frequency-based spatial-temporal attention module for

- spiking neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 22227–22235, 2025.
- Yu, X. and Al-Qaness, M. A. Human activity recognition using deep residual convolutional network based on wearable sensors. *IEEE Journal of Biomedical and Health Informatics*, 2024.
- Yuan, H., Cheng, J., Wu, Y., and Zeng, Z. Low-res mobilenet: An efficient lightweight network for low-resolution image classification in resource-constrained scenarios. *Multimedia Tools and Applications*, 81(27): 38513–38530, 2022.
- Yun, S., Han, D., Oh, S. J., Chun, S., Choe, J., and Yoo, Y. Cutmix: Regularization strategy to train strong classifiers with localizable features. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 6023–6032, 2019.
- Zhang, H., Cisse, M., Dauphin, Y. N., and Lopez-Paz, D. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=r1Ddp1-Rb>.
- Zhang, T., Xu, W., Luo, B., and Wang, G. Depth-wise convolutions in vision transformers for efficient training on small datasets. *Neurocomputing*, 617:128998, 2025.

A. Proof of Properties

This appendix provides proof and remarks of properties discussed in Section 3.

Remark on Property 3.5. We provide an intuitive explanation of how to construct the topology of FractalNet (Larsson et al., 2017) within the framework of KGN. Let $\mathcal{G}_F$ be a FractalNet with B Fractal Blocks, where each Fractal Block has C parallel columns. Then there exists a C -layer KGN $\mathcal{G}_{KGN} = (\mathcal{V}, \mathcal{E})$ that is equivalent to the FractalNet $\mathcal{G}_F$. Here we define nodes $\mathcal{V}$ of the KGN as concatenation operators (including concatenation of only one input, which degenerates to an identity map) in FractalNet, and edges $\mathcal{E}$ as single-input, single-output operators. Each edge operator represents either a convolution or an identity map. The construction of FractalNet-equivalent KGN is shown in Figure 3.

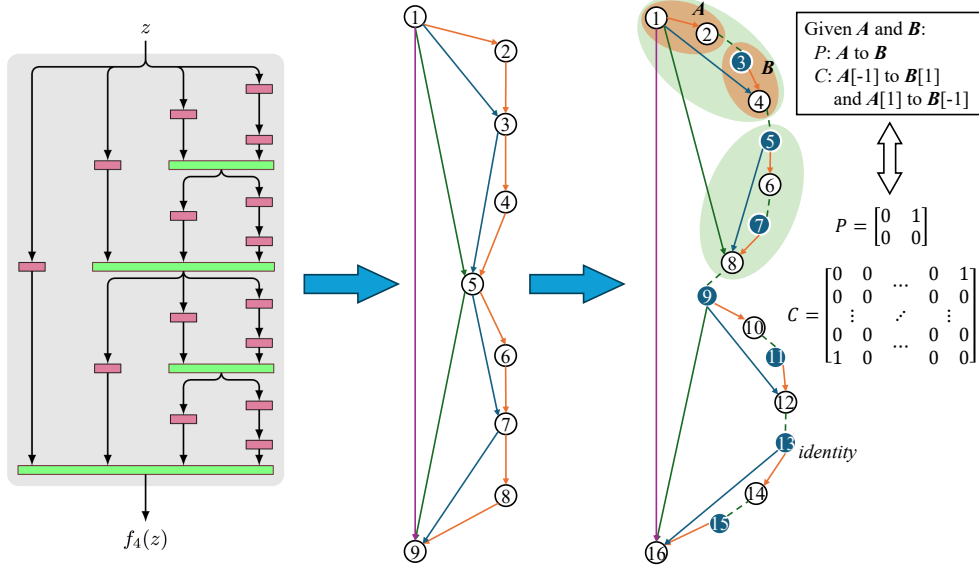


Figure 3. A KGN that is equivalent to the given FractalNet (©2017, Larsson et al.).

Approach In Figure 3, we start with the FractalNet with one Fractal Block. Based on our definition of nodes, the number of Fractal Block nodes is given by the number of concatenation operators, and the identity operators that concatenate only the single input are given by $(2^C + 1)$ (including one input node). This conversion maps the original FractalNet (left) to an abstract graph (center). Next, leave the input node and the last global concatenation node unchanged, and split each intermediate node into a function node and an identity node (nodes in blue). The function node performs the same operation as the original node before splitting, while the identity node acts as a structural placeholder. The split (right) creates $(2^C - 1)$ more nodes, which leads to $2^{C+1} \in \mathbb{Z}$ nodes in total. Let the nodes be numbered in topological order $\{1, 2, \dots, 2^{C+1}\}$, and create the 0-layer KGN abstraction by grouping each pair of the nodes:

$$V_i^0 = (2i - 1, 2i), \quad i = 1, 2, \dots, 2^C. \quad (14)$$

Each V_i represents a base pattern that connects one node to the following node, which directly implies

$$\tilde{P}_0 = P_0 = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}. \quad (15)$$

Let $(\cdot)_i$ denote the i -th element of ordered tuples. Specifically, we define $(\cdot)_{-1}$ as the last element. Next, we create the first KGN layer by connecting V_{2i-1}^0 to V_{2i}^0 , for every $i = 1, 2, \dots, 2^{C-1}$. In particular, the connections are implemented by connecting (i) $(V_{2i-1}^0)_1$ to $(V_{2i}^0)_{-1}$, and (ii) $(V_{2i-1}^0)_{-1}$ to $(V_{2i}^0)_1$. This implies

$$\tilde{P}_1 = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad C_1 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}. \quad (16)$$

Similarly, we can create the first-layer node abstraction as follows:

$$V_i^1 = (2^2i - 3, 2^2i - 2, 2^2i - 1, 2^2i), \quad i = 1, 2, \dots, 2^{C-1}. \quad (17)$$

To create the second layer, we do the same operation to the first-layer nodes: for $i = 1, 2, \dots, 2^{C-2}$, (i) connect $(V_{2i-1}^1)_1$ to $(V_{2i}^1)_{-1}$, and (ii) connect $(V_{2i-1}^1)_{-1}$ to $(V_{2i}^1)_1$. This implies

$$\tilde{P}_2 = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad C_2 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}. \quad (18)$$

Using the same approach, for the k -th layer ($0 < k \leq C$), the nodes in the $(k-1)$ -th layer are grouped by

$$V_i^{k-1} = (2^k i - j \mid j = 2^k - 1, 2^k - 2, \dots, 0), \quad i = 1, 2, \dots, 2^{C-k+1}. \quad (19)$$

And for every i , connect (i) $(V_{2i-1}^k)_1$ to $(V_{2i}^k)_{-1}$, and (ii) $(V_{2i-1}^k)_{-1}$ to $(V_{2i}^k)_1$. This implies

$$\tilde{P}_k = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad C_k = \begin{bmatrix} 0 & 0 & \dots & 0 & 1 \\ 0 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & 0 & \dots & 0 & 0 \\ 1 & 0 & \dots & 0 & 0 \end{bmatrix} \in \mathcal{M}_{\{0,1\}}^{n^k}. \quad (20)$$

Let $E_k = [e_{ij}] \in \mathcal{M}_{\{0,1\}}^{n^k}$, such that $e_{ij} = 1$ if and only if $i = 1, j = n^k$, C_k can be reformulated as $C^k = E_k + E_k^\top$.

Under this construction, properties of FractalNet, such as acyclicity, path multiplicity, and depth-wise expressiveness, follow directly from the general theory of KGNs rather than from architecture-specific arguments.

Proof of Lemma 3.6. Let C be arbitrary, P is strictly upper-triangular. According to the definition of Kronecker product, $P \otimes C$ can be expressed as

$$P \otimes C = \begin{bmatrix} O & p_{12}C & p_{13}C & \dots & p_{1n}C \\ O & O & p_{23}C & \dots & p_{2n}C \\ \vdots & \vdots & \vdots & & \vdots \\ O & O & O & \dots & p_{n-1,n}C \\ O & O & O & \dots & O \end{bmatrix}, \quad (21)$$

which is strictly block upper-triangular. □

Proof of Property 3.7. The proof is organized into sufficiency and necessity.

Sufficiency We start by showing that the two Kronecker operations in (3), $I \otimes A$ and $A \otimes C$, retain strict upper-triangularity for a square matrix $A \in \mathbb{R}^{n \times n}$, for the identity matrix I and an arbitrary matrix C of proper shapes. For $I \otimes A$, assume A is strictly upper-triangular. Let $\Sigma = I \otimes A$, define

$$\begin{cases} \tilde{i} = pn + i, & 0 \leq p \leq n-1, 1 \leq i \leq n, \\ \tilde{j} = qn + j, & 0 \leq q \leq n-1, 1 \leq j \leq n, \end{cases} \quad \text{and} \quad \mathbf{1}_{pq} = \begin{cases} 1, & p = q, \\ 0, & \text{otherwise.} \end{cases} \quad (22)$$

Therefore, $\tilde{i} \geq \tilde{j}$ if and only if $p > q$, or $p = q$ and $i \geq j$. By Kronecker product, there holds $\sigma_{\tilde{i}\tilde{j}} = \mathbf{1}_{pq}a_{ij}$. If $p > q$, $\mathbf{1}_{pq} = 0$, $\sigma_{\tilde{i}\tilde{j}} = 0$. If $p = q$ and $i \geq j$, $a_{ij} = 0$, $\sigma_{\tilde{i}\tilde{j}} = 0$. Therefore, $I \otimes A$ is strictly upper-triangular if A is strictly upper-triangular. Similarly, we can show that $A \otimes C$ is strictly upper-triangular if A is strictly upper-triangular.

Based on this observation, the KGN transform (3) yields a strictly upper-triangular result P_i when P_{i-1} and $\tilde{P}_i$ are both strictly upper-triangular. $P_0 = \tilde{P}_0$, $\tilde{P}_1 \in \tilde{\mathcal{P}}$, $P_1 = \mathcal{T}(P_0)$ is strictly upper-triangular. Assume P_{i-1} is strictly upper-triangular for any $0 < i \leq d$, since $\tilde{P}_i \in \tilde{\mathcal{P}}$ is strictly upper-triangular, $P_i = \mathcal{T}(P_{i-1})$ is also strictly upper triangular. Therefore, every P_i generated by (3) is strictly upper-triangular, and therefore nilpotent. By Lemma 3.1 and Definition 3.4, $\mathcal{G}_d$ is acyclic and topologically sorted due to nilpotent and strictly upper-triangular P_d .

Necessity Assume there exists a non-nilpotent adjacency $\tilde{P}_* \in \tilde{\mathcal{P}}, * \leq d$. Let C_* be non-nilpotent. There holds

$$P_*^k \succeq (\tilde{P}_* \otimes C_*)^k = \tilde{P}_*^k \otimes C_*^k \neq 0 \quad (23)$$

for any sufficiently large $k > 0$. Therefore, it also holds that

$$P_{*+1} \succeq (I_{n_{*+1}} \otimes P_*)^k = I_{n_{*+1}}^k \otimes P_*^k \neq 0. \quad (24)$$

Repeating (24) $(d - *)$ times, P_d is non-nilpotent. According to Lemma 3.1, KGN $\mathcal{G}_d$ with adjacency matrix P_d is cyclic, which contradicts with the statement. Therefore, when $\mathcal{G}_d$ is acyclic, none of its inducers is non-nilpotent.

This completes the proof. $\square$

Proof of Property 3.8. Statement (i) is directly implied by the shape of the Kronecker product. We prove (ii), and (iii) as follows:

(ii) Given any unweighted adjacency $P_d \in \{0, 1\}^{n_d \times n_d}$, the number of edges is the sum of its elements, i.e., $|\mathcal{E}_d| = \|P_d\|_{1,1}$. For arbitrary matrices $A, B \in \{0, 1\}^{n \times n}$, there holds

$$\|A \otimes B\|_{1,1} = \|A\|_{1,1} \times \|B\|_{1,1}. \quad (25)$$

According to (3), there exists recurrence

$$\|\mathcal{T}_i(P)\|_{1,1} = n_i \|P\|_{1,1} + \|\tilde{P}_i\|_{1,1} \|C_i\|_{1,1} = n_i \|P\|_{1,1} + N_i \quad (26)$$

for $i = 1, 2, \dots, d$. Let $x_{(i)} = \|P_i\|_{1,1}$. The recurrence can be written as

$$x_{(i+1)} - n_i x_{(i)} = N_i. \quad (27)$$

The closed-form solution of (27) is directly given by

$$|\mathcal{E}_d| = \left(\prod_{t=1}^d n_t \right) \|P_0\|_{1,1} + \sum_{i=1}^d \left(\prod_{t=i+1}^d n_t \right) N_i. \quad (28)$$

Under $\|C_0\|_{1,1} = 1$, we have $N_0 = \|P_0\|_{1,1} \|C_0\|_{1,1} = \|P_0\|_{1,1}$, statement (ii) is directly derived as

$$|\mathcal{E}_d| = \sum_{i=0}^d \left(\prod_{t=i+1}^d n_t \right) N_i. \quad (29)$$

(iii) Divide $|\mathcal{E}_d|$ by $|\mathcal{V}_d|$, there holds

$$\langle d \rangle = \sum_{i=0}^d \left(\prod_{t=i+1}^d n_t \right) N_i / \prod_{t=0}^d n_t = \sum_{i=0}^d \frac{N_i}{\prod_{t=0}^i n_t}. \quad (30)$$

This completes the proof. $\square$

Proof of Property 3.10. By the boundedness of node count and definitions in Property 3.8, there exists an upper bound $K > 0$, such that $N_i \leq K \cdot \prod_{t=1}^i n_t$ for every $i \leq d$. By Property 3.9, there holds $\langle d \rangle \leq \sum_{i=0}^d K = K(d+1)$. Based on (3), and the assumption that $P_i \in \mathbb{R}^{\geq 2 \times 2}$, there holds $|\mathcal{V}_{i+1}| \geq 2|\mathcal{V}_i|$. Therefore, we have $|\mathcal{V}_d| \geq 2^{d+1}$, or equivalently $d+1 \leq \log_2 |\mathcal{V}_d|$. Combining the above inequalities, there holds $\langle d \rangle \leq K \log_2 |\mathcal{V}_d| = P(\log n)$, where $n = |\mathcal{V}_d|$. This completes the proof. $\square$

B. Fusion Implementation

This appendix shows the detailed structures of ISF, RSF, and biased RSF, as proposed in Section 4.1. See Figure 4 and 6. In these figures, we adopt C as the number of channels, following classic deep learning studies.

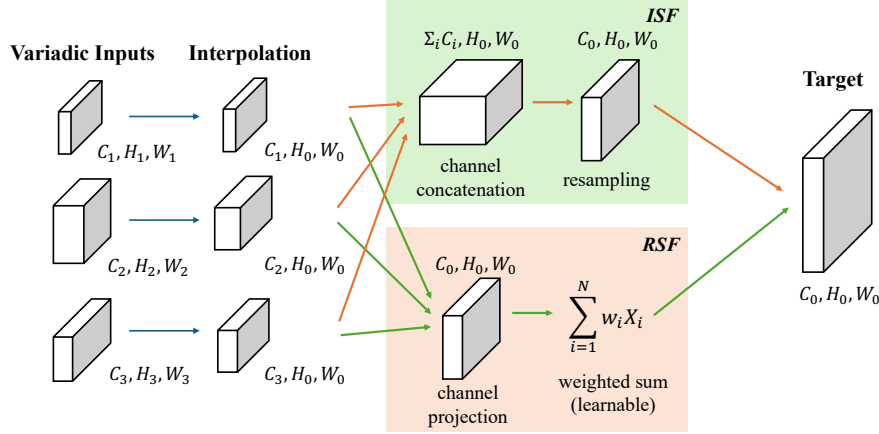


Figure 4. Implementation of ISF and RSF.

ISF The first structure is Inception-Styled Fusion (ISF), inspired by (Szegedy et al., 2015), which first resamples the input features via linear interpolation to the desired output shape, then concatenates the interpolated features along the channel dimension. The features are finally mapped to the output space with the desired number of channels. Let $\{x_i\}_{i=1}^m$ denote m inputs of different shapes, ISF is expressed by

$$\text{ISF}(x) = \text{Conv}(\text{Concat}\{f_i(x_i)\}_{i=1}^m), \quad (31)$$

where $f_i : \mathbb{R}^{n_i} \rightarrow \mathbb{R}^{n_{\text{out}}}$ denotes the linear interpolation applied to the i -th input feature. This mechanism preserves the diversity of representations across different branches, akin to multi-branch feature exploration in Inception architectures, while avoiding the heavy computational overhead of using entire multi-scale convolutional blocks. Empirically, ISF facilitates expressive mixing of information across KGN layers, enabling rich cross-path interactions without sacrificing parameter efficiency.

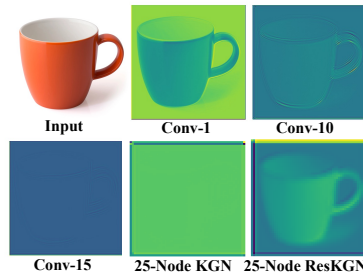


Figure 5. Local details preserved with RSF (ResKGN).

RSF The second structure is Residual-Styled Fusion (RSF). It is inspired by the well-known residual skip proposed in (He et al., 2016). In our framework, it is extended as a learnable, optionally weighted summation. The input branches are first interpolated and projected via a channel transform to the same tensor space, so that the sum can be defined over the inputs. Next, the (optionally) weighted sum is calculated over these preprocessed features. At last, these features are projected into

the desired output space via convolution. RSF is defined as follows:

$$\text{RSF}(x) = \text{Conv}_{\text{out}} \left(\sum_{i=1}^m w_i \cdot \text{Conv}_i(f_i(x)) \right), \quad (32)$$

where w_i is a learnable scaler, $f_i : \mathbb{R}^{n_i} \rightarrow \mathbb{R}^{n_{\text{out}}}$ still denotes linear interpolation for the i -th input feature. This effectively turns the fusion into a lightweight attention mechanism, where the network learns which incoming branches are more informative for the current node. Because KGN topologies can grow wide quickly, RSF provides a computationally stable and parameter-efficient way to aggregate many sources while maintaining smooth gradient propagation across the KGN hierarchy.

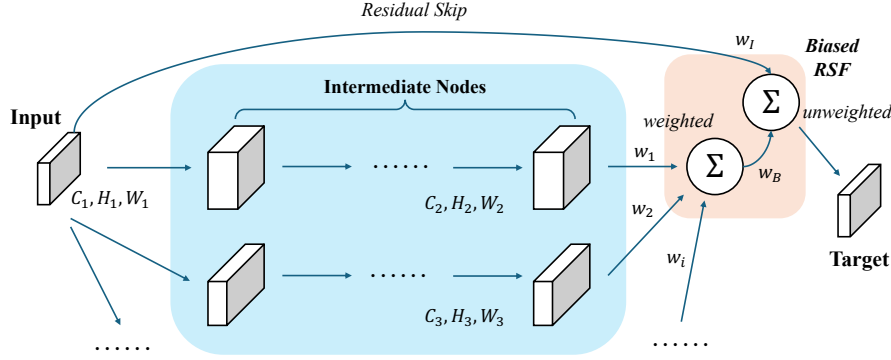


Figure 6. Implementation of Biased RSF.

Biased RSF Based on RSF, we developed a lightweight biased RSF to simulate real residual skips by implicitly amplifying the weight of a specified input. It is implemented by stacking two RSF layers. Let x_0 denote the initial input that is worth the largest weight, $\{x_i\}_{i=1}^{m-1}$ denote the rest of the inputs. Biased RSF is expressed by

$$\text{BiasedRSF} = \text{RSF}_0(x_0, \text{RSF}_1(\{x_i\}_{i=1}^{m-1})), \quad (33)$$

where RSF_0 represents an unweighted RSF, whereas RSF_1 is optionally weighted. The resulting fusion emphasizes the continuity of core representations while selectively incorporating auxiliary signals from other branches, improving optimization stability and reducing sensitivity to noisy or misaligned multi-branch inputs. In practice, Biased RSF contributes significantly to the trainability of deep KGNs, especially in preserving local details and avoiding over-smoothing (Figure 5).

Impact of Fusion Layers As discussed in Section 4.1, we employ the above fusion layers to allow the aggregation of features of different sizes. To further investigate the impact of fusion layer choices, we conduct the following ablation study on CIFAR-100, using different fusion layers while fixing both the connectivity and node planning of KGN-Helix. In particular, the KGN nodes are implemented with Conv3x3-BatchNorm to ensure controlled experiment. Ablation results are shown in Table 5.

Table 5. Fusion-level ablation results.

Fusion Type	Node Type	#Params (M)	Accuracy (Top-1)
RSF	ConvBN	0.423	71.89%
ISF	ConvBN	0.436	71.60%
Biased RSF	ConvBN	0.452	72.02%

The performance differences among the evaluated fusion layers remain relatively small despite minor variations in parameter count. This reveals that in this specific task, varying fusion operators does not lead to significant parameter overhead or accuracy drops. Thus, fusion layers are more of mechanisms allowing feature aggregation, rather than the key factor dominating model performance in these models.

C. Architecture Search

The induction of KGN's adjacency, i.e., (3), provides a rich space for neural architecture search (NAS) to be conducted systematically.

To formalize the KGN architecture search process, we define the following spaces. Let

$$\mathcal{N}_{\text{NAS}} = \{n_1, n_2, \dots, n_K\}, \quad n_i \in \mathbb{Z}_+, \quad (34)$$

denote the set of permissible node sizes for each depth. According to Property 3.8, the node count is $N_K = \prod_{i=1}^K n_i$. Let $\mathcal{V}_{\text{NAS}}$ denote a nonempty set of node functions, where each $v_i \in \mathcal{V}_{\text{NAS}}$ represents a parametrized neural module assigned to a KGN node. Let $\mathcal{A}_{\Delta}^n \subset \{0, 1\}^{n \times n}$ denote the set of all strictly upper-triangular adjacency matrices with n nodes. We further define the adjacency-search space

$$\tilde{\mathcal{P}}_{\text{NAS}}^n = \mathcal{A}_{\Delta}^n, \quad (35)$$

$$\mathcal{C}_{\text{NAS}}^n = \{0, 1\}^{n \times n}, \quad (36)$$

which contains all valid initiators $\tilde{P}_k \in \tilde{\mathcal{P}}_{\text{NAS}}^{n_k}$ and connection matrices $C_k \in \mathcal{C}_{\text{NAS}}^{n_k}$. By (3), the structure of a KGN is determined by

$$(\mathcal{V}_K, \tilde{\mathcal{P}}_K, \mathcal{C}_K) = (\{v_i\}_{i=1}^{N_K}, \{\tilde{P}_i\}_{i=0}^K, \{C_i\}_{i=1}^K), \quad (37)$$

$$v_i \in \mathcal{V}_{\text{NAS}}, \quad \tilde{P}_i \in \tilde{\mathcal{P}}_{\text{NAS}}^{n_i}, \quad C_i \in \mathcal{C}_{\text{NAS}}^{n_i}.$$

Given a depth budget $K_{\max} > 0$ and an evaluation function $\epsilon(\cdot)$, the goal of architecture search is to select

$$(\mathcal{V}_{\text{NAS}}^*, P_K^*) = \arg \max_{\mathcal{V}_K, \tilde{\mathcal{P}}_K, \mathcal{C}_K} \epsilon(\mathcal{G}), \quad K \leq K_{\max}, \quad (38)$$

where $\mathcal{G}$ denotes the KGN induced by $(\tilde{\mathcal{P}}_K, \mathcal{C}_K)$.

Algorithm 1 KGN Architecture Search

Require: maximum depth $K_{\max} > 0$, layer sizes $\mathcal{N}_{\text{NAS}}$, node-function space $\mathcal{V}_{\text{NAS}}$, adjacency spaces $\tilde{\mathcal{P}}_{\text{NAS}}^{n_k}$ and $\mathcal{C}_{\text{NAS}}^{n_k}$, evaluation function $\epsilon(\cdot)$, step budget $T_{\max} > 0$

Ensure: best KGN $\mathcal{G}_{\text{NAS}}^* = (\mathcal{V}_{\text{NAS}}^*, P_K^*)$

- 1: Initialize candidate set $\mathcal{G}_{\text{cand}} \leftarrow \emptyset$
- 2: Initialize architecture controller π_{θ}
- 3: **for** $K = 1$ **to** $K_{\max}$ **do**
- 4: Compute node count $N_K \leftarrow \prod_{i=1}^K n_i$
- 5: **for** search step $t = 1$ **to** $T_{\max}$ **do**
- 6: Sample $\mathcal{V}_K^{(t)} = \{v_i^{(t)}\}_{i=1}^{N_K}, v_i^{(t)} \sim \mathcal{V}_{\text{NAS}}$
- 7: Sample $\tilde{P}_k^{(t)} \sim \tilde{\mathcal{P}}_{\text{NAS}}^{n_k}, C_k^{(t)} \sim \mathcal{C}_{\text{NAS}}^{n_k}$, for every $k = 1, \dots, K$
- 8: Generate adjacency via (3):

$$P_k^{(t)} = I_{n_k} \otimes P_{k-1}^{(t)} + \tilde{P}_k^{(t)} \otimes C_k^{(t)}, \quad k = 1, \dots, K$$

- 9: Construct KGN $\mathcal{G}^{(t)} = (\mathcal{V}_K^{(t)}, P_K^{(t)})$
 - 10: Evaluate architecture $s^{(t)} = \epsilon(\mathcal{G}^{(t)})$
 - 11: Update controller using feedback $s^{(t)}$
 - 12: $\mathcal{G}_{\text{cand}} \leftarrow \mathcal{G}_{\text{cand}} \cup \{(\mathcal{G}^{(t)}, s^{(t)})\}$
 - 13: **end for**
 - 14: **end for**
 - 15: Select best $(\mathcal{G}_{\text{NAS}}^*, s^*) = \arg \max_{(\mathcal{G}, s) \in \mathcal{G}_{\text{cand}}} s$
-

Algorithm 1 is feasible for autonomous search. Evaluation metric ϵ can be a weighted sum of (i) topological complexity (Property 3.8), (ii) topological expressiveness (Property 3.11), and (iii) parameter count and task budget. We also adopt a

human-in-the-loop architecture selection scheme, where candidate KGNs are manually explored within a constrained search space, as discussed in Section 5.

We conduct an additional experiment to verify our NAS idea on the PAMAP2 dataset. In particular, we tested the following 6 adjacency inducers, retaining the same node implementation and training logic. The number of channels is adjusted, and some concatenation layers are replaced with 1×1 convolutional layers to ensure shape consistency during computation.

The inducers are defined and indexed as follows:

$$\tilde{P}^{(1)} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

(a) Adjacency No. 1.

$$\tilde{P}^{(2)} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

(b) Adjacency No. 2.

$$\tilde{P}^{(3)} = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

(c) Adjacency No. 3.

$$\tilde{P}^{(4)} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

(d) Adjacency No. 4.

$$\tilde{P}^{(5)} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

(e) Adjacency No. 5.

$$\tilde{P}^{(6)} = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

(f) Adjacency No. 6.

Figure 7. Adjacency inducers for additional architecture search task.

The i -th candidate network adopts $\tilde{P}_0^{(i)} = \tilde{P}_1^{(i)} = \tilde{P}^{(i)}$ as adjacency inducers (see (3)). The connection matrices are defined in the same way as the KGN-PAMAP2 model. We adopt the identical training settings for the above network candidates. The test results on PAMAP2 are given in Table 6, where the result of full KGN-PAMAP2 is bolded. We also show the relationship between accuracy and the number of parameters of candidates in Figure 8.

Table 6. Comparison of test accuracy of network candidates.

Group	Variant	Accuracy	#Params
Ablation	Without KGN	84.8%	6,540
	Chain topology	89.9%	171,596
KGN	Full KGN-PAMAP2	95.6%	205,644
	Using inducer 1	95.2%	198,796
	Using inducer 2	95.0%	197,964
	Using inducer 3	95.0%	194,892
	Using inducer 4	94.3%	192,332
	Using inducer 5	95.2%	188,044
	Using inducer 6	91.7%	187,212

According to Table 6, there exists a significant gap between KGNs with inducers that are sparse at different levels. Our full KGN-PAMAP2, the most dense variant among all candidates, achieves the highest accuracy of 95.6%. Candidates with inducers that remove specific branches achieve accuracy scores ranging from 91.7% to 95.2%, with slightly fewer parameters. In particular, the last candidate (No. 6) achieves the least accuracy score of 91.7%, which is closer to the chain variant in our ablation studies (achieving 89.9%, cited in Table 4) than other variants of KGN.

Together, these results suggest that the complexity of the model topology influences the accuracy of PAMAP2 human activity recognition, and our full KGN-PAMAP2 implementation achieves the highest accuracy among these candidates. At the same time, Figure 8 shows that while the fifth candidate has a similar number of parameters to the sixth, its accuracy score is significantly lower. This also suggests that the connections between nodes are an important factor in determining model performance.

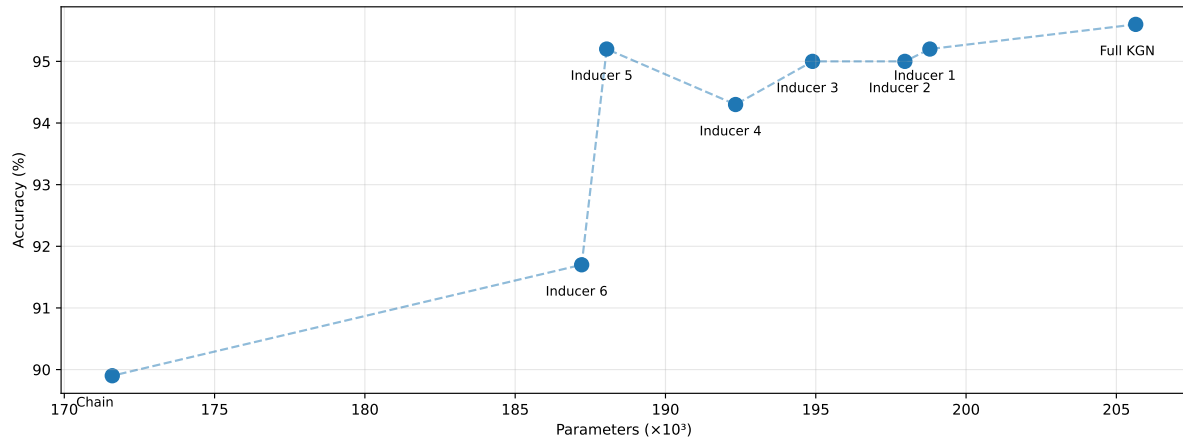


Figure 8. Relationship between accuracy scores and parameter count.

D. Node Implementation

This appendix provides node planning and implementation details for reference and replication. We also adopt C as the number of channels, K as the size of kernels, following classic deep learning studies. Figure 9 shows the connections of KGN-Helix5 (series), KGN-HAR, and KGN-PAMAP2.

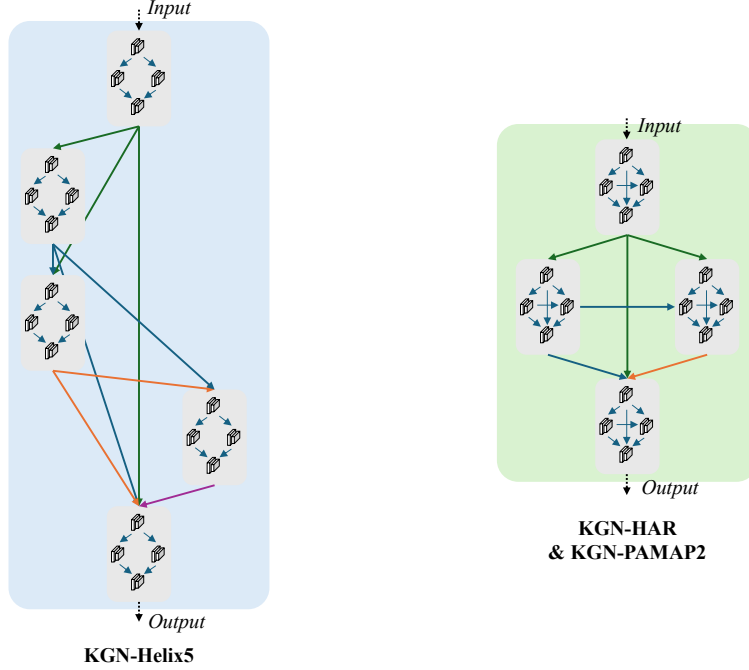


Figure 9. Visualization of topology of KGN-Helix5, KGN-HAR and KGN-PAMAP2.

D.1. Helix-5

An SCEF node is shown in Figure 10.

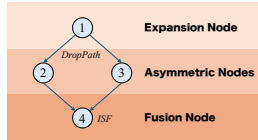


Figure 10. Three-level structure of SCEF nodes.

Node planning used by the proposed KGN-Helix5 architecture is shown in Table 7.

D.2. Atomic Node Design

The atomic node serves as the fundamental computational unit in KGN-PAMAP2. It is designed as a lightweight yet expressive temporal convolutional block, inspired by inverted bottleneck structures while remaining fully deterministic and topology-agnostic. Each atomic node consists of three stages.

Expansion First, an expansion layer applies a 1×1 pointwise convolution to increase the channel dimension by a factor of *expand_ratio*, followed by batch normalization and a SiLU activation. This expansion enlarges the intermediate representation space at low computational cost.

Convolution Second, a depthwise convolution is applied independently to each expanded channel. This operation supports configurable kernel size and dilation, enabling local or dilated receptive fields without introducing cross-channel mixing.

Kronecker Generative Network

Table 7. Helix5 node planning (level-2).

Variant	Component	Indices	Node Type	C_{in}	C_{out}	K_1	K_2	K_3	K_4	Pooling	DropPath
Helix5-C10-S	KGN Encoder	1	Input	3	48	3	5				0.00
		2	SCEF	48	64	3	3	5	3		0.01
		3	SCEF	112	96	3	3	7	3	✓	0.02
		4	SCEF	160	128	3	3	7	5	✓	0.03
		5	Pymarid Fusion	432	112					✓	0.00
Helix5-C100 Helix5-C10-L	KGN Encoder	1	Input	3	64	3	5				0.00
		2	SCEF	64	64	3	3	5	3		0.02
		3	SCEF	128	96	3	3	7	3	✓	0.03
		4	SCEF	160	112	3	3	7	5	✓	0.04
		5	Pymarid Fusion	480	128					✓	0.00
All variants	Decoder	–	Linear	–	–						0.15 (dropout)

Batch normalization and SiLU activation are applied to stabilize training. Third, a projection layer compresses the expanded representation back to the target output dimension using another 1×1 convolution, followed by batch normalization. When enabled, a residual skip connection aligns the input and output channels via a 1×1 convolution and is added to the output.

Activation Finally, the output is passed through a SiLU activation and Dropout1d for regularization. Dropout is applied after feature projection rather than within the convolutional stages, which empirically improves robustness on noisy sequence data such as PAMAP2.

D.3. KGN-HAR

Node planning for KGN-HAR is summarized in Table 8. Each pattern follows the same internal DAG structure with scale-dependent output length.

Table 8. KGN-HAR node planning (pattern-level DAG).

Pattern	Node	Type	C_{in}	C_{out}	Kernel	Dilation	Stride	Skip	Dropout
KGNPattern	1	Atomic	32	32	3	1	1	✓	0.20
	2	Atomic	32	32	3	1	1	–	0.00
	3	Atomic	48	64	3	2	1	–	0.20
	4	Atomic	48	64	5	1	2	✓	0.20
<i>Fusion: Inception-style concatenation followed by 1×1 projection</i>									

D.4. KGN-PAMAP2

KGN-PAMAP2 adopts a robust inverted-bottleneck atomic node. Node configurations are shown in Table 9.

Table 9. KGN-PAMAP2 atomic node design.

Node	C_{in}	C_{out}	Kernel	Dilation	Expand	Skip	Dropout
Node 1	32	32	3	1	2	✓	0.1
Node 2	32	48	5	1	2	–	0.1
Node 3	32	48	3	2	2	–	0.1
Node 4	48	64	3	1	3	✓	0.1

E. Average In-Degree Analysis

In this section, we analyze the average in-degrees of FractalNet (Larsson et al., 2017), DenseNet (Huang et al., 2017), and KGN. According to Larsson et al., the number of atomic convolutional blocks of FractalNet is given by $N_{\text{node}}^{\text{F}} = B \cdot 2^{C-1}$, where B represents the number of Fractal Blocks, C denotes the number of parallel columns within each Fractal Block. Using the fractal expansion rule, we can deduce that the number of edges per Fractal Block satisfies

$$\tilde{N}_{\text{edge}}^{\text{F}} = \sum_{c=1}^C 2^c = 2(2^C - 1). \quad (39)$$

This implies that the total number of edges satisfies

$$N_{\text{edge}}^{\text{F}} = B \tilde{N}_{\text{edge}}^{\text{F}} = 2B(2^C - 1). \quad (40)$$

Therefore, the average in-degree of FractalNet is computed as

$$\langle d \rangle_{\text{F}} = \frac{2B(2^C - 1)}{B \cdot 2^{C-1}} = 4 \left(1 - \frac{1}{2^C} \right) = O(1). \quad (41)$$

Similarly, based on the structure of DenseNet, let $N_{\text{node}}^{\text{D}}$ denote its node count. The number of edges is given by

$$N_{\text{edge}}^{\text{D}} = \sum_{i=1}^{N_{\text{node}}^{\text{D}}-1} i = \frac{1}{2} N_{\text{node}}^{\text{D}} (N_{\text{node}}^{\text{D}} - 1). \quad (42)$$

Divide $N_{\text{edge}}^{\text{D}}$ by $N_{\text{node}}^{\text{D}}$, there holds

$$\langle d \rangle_{\text{D}} = \frac{1}{2} (N_{\text{node}}^{\text{D}} - 1) = O(N_{\text{node}}^{\text{D}}). \quad (43)$$

Remark Since both FractalNet and DenseNet can be considered as a realization of KGN, their average in-degrees can also be derived from Property 3.8.

FractalNet. Based on the analysis and definitions in Appendix A on Property 3.5, there holds

$$n_k \equiv 2, \quad \|\tilde{P}_k\|_{1,1} \equiv 1, \quad \|C_k\|_{1,1} = N_k = \begin{cases} 1, & \text{if } k = 0, \\ 2, & \text{otherwise.} \end{cases} \quad (44)$$

Using Statement (iii), for a FractalNet with one FractalBlock, and $C > 1$ columns, there holds

$$\langle d \rangle_{\text{F}} = \sum_{k=0}^C \frac{N_k}{\prod_{t=0}^k n_t} = \frac{1}{2} + \sum_{k=1}^C \frac{2}{2^{k+1}} = \frac{1}{2} + \frac{2^{-1}(1 - 2^{-C})}{1 - 2^{-1}} = \frac{3}{2} - 2^{-C} = O(1). \quad (45)$$

We note that the closed-form expression for the average in-degree differs from our previous result because the two results use different definitions of nodes and edges (i.e., the node split). To verify this result, we can refer to Figure 3, where there are 22 edges and 16 nodes. Let $C = 3$, the average in-degree is then computed as $3/2 - 2^{-3} = 22/16$, which satisfies our observation. Despite this distinction, the growth rate of the average in-degree of FractalNet is still $O(1)$, which matches our previous conclusion.

DenseNet. Using Statement (iii), let $d = 0$, $n_0 = N_{\text{node}}^{\text{D}}$, there holds

$$\|P_0\|_{1,1} = \frac{1}{2} n_0 (n_0 - 1) \Rightarrow \langle d \rangle_{\text{D}} = \frac{\|P_0\|_{1,1}}{n_0} = \frac{1}{2} (n_0 - 1) = \frac{1}{2} (N_{\text{node}}^{\text{D}} - 1). \quad (46)$$

This conclusion is equivalent to our earlier result.

F. ImageNet Training

This appendix provides details of a supplementary experiment that trains and evaluates KGN with ImageNet-1K (Deng et al., 2009) dataset (i.e., ILSVRC2012). ImageNet-1K dataset contains 1281167 images with labels for training, and 50000 for validation. Both training and validation images are grouped into 1000 categories. Due to time constraint, we train our model for 223 epochs, recording both validation loss values and the top-1 accuracy during the training process, and evaluating the top-5 accuracy at the 214th epoch, which saves the best checkpoint throughout the training process.

Training settings We follow the official PyTorch training pipeline, adopting standard data augmentation including random resized cropping (224×224), horizontal flipping and color jitter for training; for validation, we only adopt fixed resizing and center-cropping (224×224). Images are also normalized with the conventional per-channel mean-std statistics, as shown below:

$$\text{mean} = (0.485, 0.456, 0.406), \quad \text{std} = (0.229, 0.224, 0.225).$$

As for the training utilities, we adopt AdamW (Loshchilov & Hutter, 2019) as the optimizer, and a cosine learning rate scheduler (Loshchilov & Hutter, 2017). The base learning rate is 0.001, and the batch size is 64. During the training process, we adopt cross entropy as the loss function.

Model design In this experiment, we still adopt the KGN-Helix5 architecture (see Figure 9), with different implementation of atomic nodes (see Table 10). Since the size of images of ImageNet dataset is much larger than that of CIFAR datasets, we reduce the feature sizes node by node using average pooling.

Table 10. Specification of atomic nodes.

Operator	Kernel	Stride	Dilation	Groups	In Ch.	Out Ch.
Expand (PW)	1×1	1	1	g_{exp}	C_{in}	$C_{\text{mid}} = C_{\text{in}} \cdot r$
Depthwise Conv	$k \times k$	1	d	C_{mid}	C_{mid}	C_{mid}
Project (PW)	1×1	1	1	g_{proj}	C_{mid}	C_{out}
Residual	1×1	1	1	1	C_{in}	C_{out}
BN / Act / Drop	–	–	–	–	C_{out}	C_{out}

Table 11. Specification of KGN patterns.

Operator	Kernel	Stride	Dilation	Groups	In Ch.	Out Ch.	Notes
Node ₁ : Expand	1×1	1	1	g_{exp}	C_{in}	$C_1 r_1$	–
Node ₁ : DW	3×3	1	1	$C_1 r_1$	$C_1 r_1$	$C_1 r_1$	–
Node ₁ : Project	1×1	1	1	g_{proj}	$C_1 r_1$	C_1	skip
Node ₂ : Expand	1×1	1	1	g_{exp}	C_1	$C_2 r_2$	–
Node ₂ : DW	5×5	1	1	$C_2 r_2$	$C_2 r_2$	$C_2 r_2$	–
Node ₂ : Project	1×1	1	1	g_{proj}	$C_2 r_2$	C_2	no skip
Fuse ₁₂ (Cat+PW)	1×1	1	1	g_{f}	$C_1 + C_2$	C_2	–
Node ₃ : Expand	1×1	1	1	g_{exp}	C_2	$C_3 r_3$	–
Node ₃ : DW	3×3	1	2	$C_3 r_3$	$C_3 r_3$	$C_3 r_3$	–
Node ₃ : Project	1×1	1	1	g_{proj}	$C_3 r_3$	C_3	no skip
Fuse ₁₂₃ (Cat+PW)	1×1	1	1	g_{f}	$C_1 + C_2 + C_3$	C_3	–
Node ₄ : Expand	1×1	1	1	g_{exp}	C_3	$C_4 r_4$	–
Node ₄ : DW	3×3	1	1	$C_4 r_4$	$C_4 r_4$	$C_4 r_4$	–
Node ₄ : Project	1×1	1	1	g_{proj}	$C_4 r_4$	C_4	skip

The default values of g_{exp} and g_{proj} are 1. Based on atomic nodes, the patterns are arranged according to the Helix5 architecture, as shown in Table 11 ($g_{\text{f}} = 1$). Each pattern forms a depth-1 KGN, while the combination of 5 patterns forms the full depth-2 KGN.

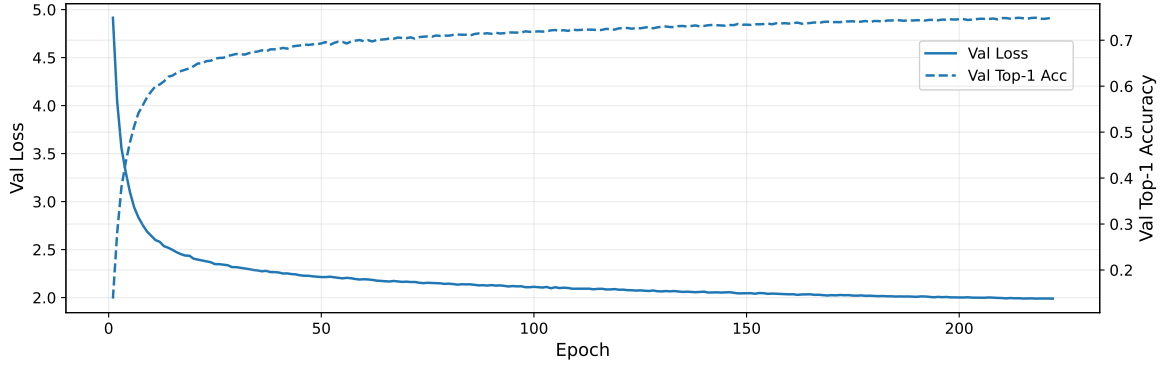


Figure 11. Loss and accuracy variation during the training of KGN on ImageNet.

Table 12. Comparison of model sizes and accuracy on ImageNet-1K dataset.

Group	Variant	#Params	Top-1 Acc.	Top-5 Acc.
Spiking Transformer (Guo et al., 2025)	ST-8-384	16.81M	74.04%	–
DenseNet (Huang et al., 2017)	DenseNet-121 ^a	7.979M	74.434%	91.972%
CondenseNet (Huang et al., 2018)	CondenseNet ($G = C = 8$)	2.9M	71.0%	90.0%
	CondenseNet ($G = C = 4$)	4.8M	73.8%	91.7%
MobileNet	MobileNetV1 (Howard et al., 2017)	4.2M	70.6%	–
	MobileNetV2 (Sandler et al., 2018)	3.4M	72.0%	–
	MobileNetV2 (1.4) (Sandler et al., 2018)	6.9M	74.7%	–
KGN (Ours)	Helix5-ImageNet	4.627M	74.89%	92.23%

^a The ImageNet accuracy of DenseNet-121 is taken from the official PyTorch model repository (<https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.densenet121.html>) because the original paper (Huang et al., 2017) does not provide explicit #params, Top-1/Top-5 accuracy report under the configuration.

Training results Our model reaches a top-1 accuracy of 74.89% on ImageNet-1K, obtained at epoch 214 (see Figure 11). The model contains only 4.63M parameters and requires 2.18 GFLOPs per 224×224 forward pass, placing it at significantly lower computational and parameter cost compared to DenseNet-121 (Huang et al., 2017) while achieving competitive accuracy under identical training conditions. Throughout training, validation accuracy improves steadily during the first 200 epochs and saturates near 75%, with no signs of divergence, instability, or irregular curvature.

Comparison Across a range of compact ImageNet baselines, our model achieves the strongest accuracy with acceptable price of parameter count. MobileNetV1 and MobileNetV2 obtain 70% to 74% top-1 accuracy with 3 to 7M parameters, while CondenseNet reaches 71% to 74% using 2.9 to 4.8M parameters. DenseNet-121 provides a stronger reference point at 74.43% top-1 and 91.97% top-5 accuracy with 7.98M parameters, and recent spiking-transformer models achieve comparable accuracy only at substantially larger scale. In contrast, our KGN model attains 74.89% top-1 and 92.23% top-5 accuracy using only 4.63M parameters, surpassing models of similar size and approaching or exceeding the performance of significantly larger architectures. These results indicate that KGN topologies offer competitive representational capacity while remaining markedly more parameter-efficient.